

```
!nvidia-smi
```

```
Sun Feb 1 11:35:56 2026
```

```
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|   0   Tesla T4             Off      | 00000000:00:04:0 Off  |            0         |
| N/A   55C    P8             10W /  70W |  0MiB / 15360MiB     |      0%    Default   |
|                                           | N/A               |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| Processes:                         GPU Memory |
| GPU   GI    CI        PID   Type   Process name                      Usage |
| ID   ID                               |                          |
+-----+-----+-----+-----+-----+-----+
| No running processes found          |
+-----+-----+-----+-----+-----+-----+
```

```
!pip -q install nvcc4jupyter
%load_ext nvcc4jupyter
```

```
Detected platform "Colab". Running its setup...
Source files will be saved in "/tmp/tmpvae1yg38".
```

```
%%writefile overlap.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <vector>
#include <chrono>
#include <algorithm>
#include <iostream>

#define CHECK_CUDA(call) do {
    cudaError_t err = (call);
    if (err != cudaSuccess) {
        fprintf(stderr, "CUDA error %s:%d: %s\n",
            __FILE__, __LINE__, cudaGetErrorString(err));
        std::exit(1);
    }
} while(0)
```

```

static inline int div_up(int a, int b) { return (a + b - 1) / b; }

__global__ void heavy_elementwise(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  int n, int iters)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    float x = in[idx];
    // Управляемая вычислительная нагрузка: чем больше iters, тем тяжелее kernel
    #pragma unroll 1
    for (int i = 0; i < iters; ++i) {
        // Несколько FMA-подобных операций
        x = x * 1.000000119f + 0.000000119f;
        x = x * 0.999999881f - 0.000000059f;
    }
    out[idx] = x;
}

struct Args {
    size_t bytes = 512ull * 1024ull * 1024ull; // 512 MB
    size_t chunk_bytes = 32ull * 1024ull * 1024ull; // 32 MB
    int streams = 4;
    int iters = 256;
    int warmup = 1;
    int reps = 5;
    int pinned = 1;    // 1 = cudaMallocHost, 0 = malloc
    int verify = 1;    // 1 = проверка результата (быстрая)
};

static void print_usage() {
    std::cout
        << "Usage: ./overlap [options]\n"
        << "Options:\n"
        << "  --bytes <N>          total bytes (default 536870912)\n"
        << "  --chunk <N>          chunk bytes (default 33554432)\n"
        << "  --streams <N>        number of streams (default 4)\n"
        << "  --iters <N>          kernel iters per element (default 256)\n"
        << "  --warmup <N>         warmup runs (default 1)\n"
        << "  --reps <N>           measured reps (default 5)\n"
        << "  --pinned <0|1>       host pinned memory (default 1)\n"
        << "  --verify <0|1>       verify output (default 1)\n";
}

static Args parse_args(int argc, char** argv) {
    Args a;
    for (int i = 1; i < argc; ++i) {
        std::string k = argv[i];
        auto need = [&](const char* name) {

```

```

        if (i + 1 >= argc) {
            std::cerr << "Missing value for " << name << "\n";
            std::exit(1);
        }
    };
    if (k == "--bytes") { need("--bytes"); a.bytes = std::stoull(argv[++i]); }
    else if (k == "--chunk") { need("--chunk"); a.chunk_bytes = std::stoull(argv[++i]); }
    else if (k == "--streams") { need("--streams"); a.streams = std::stoi(argv[++i]); }
    else if (k == "--iters") { need("--iters"); a.iters = std::stoi(argv[++i]); }
    else if (k == "--warmup") { need("--warmup"); a.warmup = std::stoi(argv[++i]); }
    else if (k == "--reps") { need("--reps"); a.reps = std::stoi(argv[++i]); }
    else if (k == "--pinned") { need("--pinned"); a.pinned = std::stoi(argv[++i]); }
    else if (k == "--verify") { need("--verify"); a.verify = std::stoi(argv[++i]); }
    else if (k == "--help" || k == "-h") { print_usage(); std::exit(0); }
    else {
        std::cerr << "Unknown option: " << k << "\n";
        print_usage();
        std::exit(1);
    }
}

if (a.chunk_bytes == 0 || a.chunk_bytes > a.bytes) a.chunk_bytes = a.bytes;
if (a.streams <= 0) a.streams = 1;
if (a.iters < 0) a.iters = 0;
if (a.reps <= 0) a.reps = 1;
if (a.warmup < 0) a.warmup = 0;
if (a.pinned != 0 && a.pinned != 1) a.pinned = 1;
if (a.verify != 0 && a.verify != 1) a.verify = 1;
return a;
}

static void fill_input(float* h, int n) {
    for (int i = 0; i < n; ++i) {
        // детерминированные данные
        h[i] = 0.001f * (float)(i % 1024) + 1.0f;
    }
}

static bool quick_verify(const float* ref, const float* got, int n) {
    // Быстрая проверка: сравним несколько точек + max abs diff
    int probes[] = {0, n/7, n/3, n/2, (int)(0.9*n), n-1};
    float max_diff = 0.0f;
    for (int j = 0; j < (int)(sizeof(probes)/sizeof(probes[0])); ++j) {
        int i = std::max(0, std::min(n-1, probes[j]));
        float d = std::abs(ref[i] - got[i]);
        max_diff = std::max(max_diff, d);
    }
    // Если n небольшой — ужесточим
    if (n < 1'000'000) {
        for (int i = 0; i < n; i += std::max(1, n/1024)) {
            float d = std::abs(ref[i] - got[i]);
            max_diff = std::max(max_diff, d);
        }
    }
}

```

```

    }
}
std::cout << "Verify max abs diff (probes): " << max_diff << "\n";
return max_diff < 1e-3f; // допуск для наших операций
}

static void cpu_reference(const float* in, float* out, int n, int iters) {
    for (int idx = 0; idx < n; ++idx) {
        float x = in[idx];
        for (int i = 0; i < iters; ++i) {
            x = x * 1.000000119f + 0.000000119f;
            x = x * 0.999999881f - 0.000000059f;
        }
        out[idx] = x;
    }
}

static double ms_since(const std::chrono::steady_clock::time_point& t0,
                      const std::chrono::steady_clock::time_point& t1) {
    return std::chrono::duration<double, std::milli>(t1 - t0).count();
}

struct TimingSplit {
    float h2d_ms = 0.0f;
    float kernel_ms = 0.0f;
    float d2h_ms = 0.0f;
};

// Sync: H2D (cudaMemcpy) + kernel + D2H (cudaMemcpy) в одном потоке
static double run_sync(const Args& a, float* h_in, float* h_out,
                      float* d_in, float* d_out,
                      TimingSplit* split_out)
{
    const int n = (int)(a.bytes / sizeof(float));
    const int block = 256;
    const int grid = div_up(n, block);

    cudaEvent_t e0, e1, e2, e3;
    CHECK_CUDA(cudaEventCreate(&e0));
    CHECK_CUDA(cudaEventCreate(&e1));
    CHECK_CUDA(cudaEventCreate(&e2));
    CHECK_CUDA(cudaEventCreate(&e3));

    auto t0 = std::chrono::steady_clock::now();

    CHECK_CUDA(cudaEventRecord(e0, 0));
    CHECK_CUDA(cudaMemcpy(d_in, h_in, a.bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaEventRecord(e1, 0));

    heavy_elementwise<<<grid, block>>>(d_in, d_out, n, a.iters);
    CHECK_CUDA(cudaGetLastError());

```

```

CHECK_CUDA(cudaEventRecord(e2, 0));

CHECK_CUDA(cudaMemcpy(h_out, d_out, a.bytes, cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaEventRecord(e3, 0));

CHECK_CUDA(cudaDeviceSynchronize());

auto t1 = std::chrono::steady_clock::now();

float h2d=0, k=0, d2h=0;
CHECK_CUDA(cudaEventElapsedTime(&h2d, e0, e1));
CHECK_CUDA(cudaEventElapsedTime(&k, e1, e2));
CHECK_CUDA(cudaEventElapsedTime(&d2h, e2, e3));
if (split_out) { split_out->h2d_ms = h2d; split_out->kernel_ms = k; split_out->d2h_ms = d2h; }

CHECK_CUDA(cudaEventDestroy(e0));
CHECK_CUDA(cudaEventDestroy(e1));
CHECK_CUDA(cudaEventDestroy(e2));
CHECK_CUDA(cudaEventDestroy(e3));

return ms_since(t0, t1);
}

// Async: чанки + несколько stream, в каждом: H2D async -> kernel -> D2H async
static double run_async(const Args& a, float* h_in, float* h_out)
{
    const size_t total_bytes = a.bytes;

    const size_t chunk_bytes = a.chunk_bytes;
    const int n_chunks = (int)div_up((int)total_bytes, (int)chunk_bytes);

    const int nStreams = a.streams;
    std::vector<cudaStream_t> streams(nStreams);
    for (int s = 0; s < nStreams; ++s) CHECK_CUDA(cudaStreamCreate(&streams[s]));

    // Per-stream device buffers (double-buffering across streams)
    std::vector<float*> d_in_s(nStreams, nullptr);
    std::vector<float*> d_out_s(nStreams, nullptr);
    for (int s = 0; s < nStreams; ++s) {
        CHECK_CUDA(cudaMalloc(&d_in_s[s], chunk_bytes));
        CHECK_CUDA(cudaMalloc(&d_out_s[s], chunk_bytes));
    }

    const int block = 256;

    auto t0 = std::chrono::steady_clock::now();

    for (int c = 0; c < n_chunks; ++c) {
        const int s = c % nStreams;

```

```

const size_t off_bytes = (size_t)c * chunk_bytes;
const size_t cur_bytes = std::min(chunk_bytes, total_bytes - off_bytes);
const int cur_elems = (int)(cur_bytes / sizeof(float));

float* h_in_ptr = (float*)((unsigned char*)h_in + off_bytes);
float* h_out_ptr = (float*)((unsigned char*)h_out + off_bytes);

CHECK_CUDA(cudaMemcpyAsync(d_in_s[s], h_in_ptr, cur_bytes,
                           cudaMemcpyHostToDevice, streams[s]));

const int grid = div_up(cur_elems, block);
heavy_elementwise<<<grid, block, 0, streams[s]>>>(d_in_s[s], d_out_s[s], cur_elems, a.iters);
CHECK_CUDA(cudaGetLastError());

CHECK_CUDA(cudaMemcpyAsync(h_out_ptr, d_out_s[s], cur_bytes,
                           cudaMemcpyDeviceToHost, streams[s]));
}

CHECK_CUDA(cudaDeviceSynchronize());

auto t1 = std::chrono::steady_clock::now();

for (int s = 0; s < nStreams; ++s) {
    CHECK_CUDA(cudaFree(d_in_s[s]));
    CHECK_CUDA(cudaFree(d_out_s[s]));
    CHECK_CUDA(cudaStreamDestroy(streams[s]));
}

return ms_since(t0, t1);
}

int main(int argc, char** argv)
{
    Args a = parse_args(argc, argv);

    int dev = 0;
    CHECK_CUDA(cudaSetDevice(dev));
    cudaDeviceProp prop{};
    CHECK_CUDA(cudaGetDeviceProperties(&prop, dev));

    std::cout << "GPU: " << prop.name << "\n";
    std::cout << "Total bytes: " << a.bytes << " (" << (double)a.bytes/1024.0/1024.0 << " MiB)\n";
    std::cout << "Chunk bytes: " << a.chunk_bytes << " (" << (double)a.chunk_bytes/1024.0/1024.0 << " MiB)\n";
    std::cout << "Streams: " << a.streams << "\n";
    std::cout << "Kernel iters: " << a.iters << "\n";
    std::cout << "Host pinned: " << a.pinned << "\n";
    std::cout << "Warmup: " << a.warmup << ", reps: " << a.reps << "\n";

    const int n = (int)(a.bytes / sizeof(float));

    // Host buffers

```

```

float* h_in = nullptr;
float* h_out_sync = nullptr;
float* h_out_async = nullptr;

if (a.pinned) {
    CHECK_CUDA(cudaMallocHost(&h_in, a.bytes));
    CHECK_CUDA(cudaMallocHost(&h_out_sync, a.bytes));
    CHECK_CUDA(cudaMallocHost(&h_out_async, a.bytes));
} else {
    h_in = (float*)std::malloc(a.bytes);
    h_out_sync = (float*)std::malloc(a.bytes);
    h_out_async = (float*)std::malloc(a.bytes);
    if (!h_in || !h_out_sync || !h_out_async) {
        std::cerr << "malloc failed\n";
        return 1;
    }
}

fill_input(h_in, n);
std::memset(h_out_sync, 0, a.bytes);
std::memset(h_out_async, 0, a.bytes);

// Device buffers for sync run
float* d_in = nullptr;
float* d_out = nullptr;
CHECK_CUDA(cudaMalloc(&d_in, a.bytes));
CHECK_CUDA(cudaMalloc(&d_out, a.bytes));

// Warmup
for (int i = 0; i < a.warmup; ++i) {
    TimingSplit tmp{};
    (void)run_sync(a, h_in, h_out_sync, d_in, d_out, &tmp);
    (void)run_async(a, h_in, h_out_async);
}

// Measured runs
std::vector<double> sync_ms(a.reps), async_ms(a.reps);
TimingSplit last_split{};

for (int r = 0; r < a.reps; ++r) {
    TimingSplit split{};
    sync_ms[r] = run_sync(a, h_in, h_out_sync, d_in, d_out, &split);
    last_split = split;
}

for (int r = 0; r < a.reps; ++r) {
    async_ms[r] = run_async(a, h_in, h_out_async);
}

auto median = [](std::vector<double> v) {
    std::sort(v.begin(), v.end());
    return v[v.size()/2];
};

```

```

};

const double sync_med = median(sync_ms);
const double async_med = median(async_ms);

std::cout << "\n=== Results (median over reps) ===\n";
std::cout << "Sync end-to-end (ms): " << sync_med << "\n";
std::cout << " Split approx (ms): H2D " << last_split.h2d_ms
    << " | Kernel " << last_split.kernel_ms
    << " | D2H " << last_split.d2h_ms << "\n";
std::cout << "Async end-to-end (ms): " << async_med << "\n";

double speedup = sync_med / async_med;
std::cout << "Speedup (sync/async): " << speedup << "x\n";

// Verification (опционально): сравним sync и async результаты,
// и (если включено) сверим с CPU на небольшой подвыборке.
if (a.verify) {
    // Быстрая sanity-проверка: sync vs async
    bool ok_sa = quick_verify(h_out_sync, h_out_async, n);
    std::cout << "Verify sync vs async: " << (ok_sa ? "OK" : "FAIL") << "\n";

    // CPU reference может быть очень медленным на огромных данных.
    // Сделаем CPU reference на первых M элементах (без "плейсхолдеров": M фиксируем по разумной границе).
    const int M = std::min(n, 1'000'000); // до 1e6 элементов
    std::vector<float> cpu_out(M);
    cpu_reference(h_in, cpu_out.data(), M, a.itors);

    bool ok_cpu = quick_verify(cpu_out.data(), h_out_sync, M);
    std::cout << "Verify CPU vs sync (first " << M << " elems): " << (ok_cpu ? "OK" : "FAIL") << "\n";
}

CHECK_CUDA(cudaFree(d_in));
CHECK_CUDA(cudaFree(d_out));

if (a.pinned) {
    CHECK_CUDA(cudaFreeHost(h_in));
    CHECK_CUDA(cudaFreeHost(h_out_sync));
    CHECK_CUDA(cudaFreeHost(h_out_async));
} else {
    std::free(h_in);
    std::free(h_out_sync);
    std::free(h_out_async);
}

return 0;
}

```



```
!nvcc -O3 -std=c++17 overlap.cu -o overlap \
-gencode arch=compute_75,code=sm_75
```

```
!./overlap --bytes 536870912 --chunk 33554432 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 1
```

```
GPU: Tesla T4
Total bytes: 536870912 (512 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 108.023
  Split approx (ms): H2D 43.9365 | Kernel 23.2468 | D2H 40.8313
Async end-to-end (ms): 58.113
Speedup (sync/async): 1.85884x
Verify max abs diff (probes): 0
Verify sync vs async: OK
Verify max abs diff (probes): 3.05176e-05
Verify CPU vs sync (first 1000000 elems): OK
```

Pinned ON:

```
!./overlap --bytes 1073741824 --chunk 33554432 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
```

```
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 222.319
  Split approx (ms): H2D 88.1059 | Kernel 52.2526 | D2H 82.2299
Async end-to-end (ms): 116.935
Speedup (sync/async): 1.90121x
```

Pinned OFF:

```
!./overlap --bytes 1073741824 --chunk 33554432 --streams 4 --iters 256 --pinned 0 --warmup 1 --reps 5 --verify 0
```

```
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 0
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 536.194
  Split approx (ms): H2D 225.732 | Kernel 50.5213 | D2H 226.017
Async end-to-end (ms): 504.888
Speedup (sync/async): 1.06201x
```

Chank affect:

```
!./overlap --bytes 1073741824 --chunk 8388608 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
!./overlap --bytes 1073741824 --chunk 16777216 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
!./overlap --bytes 1073741824 --chunk 33554432 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
!./overlap --bytes 1073741824 --chunk 67108864 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
```

```
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 8388608 (8 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 221.058
  Split approx (ms): H2D 87.7823 | Kernel 50.3825 | D2H 81.6446
Async end-to-end (ms): 120.188
Speedup (sync/async): 1.83928x
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 16777216 (16 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 220.394
  Split approx (ms): H2D 87.628 | Kernel 50.2504 | D2H 81.6327
Async end-to-end (ms): 118.86
Speedup (sync/async): 1.85422x
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 4
Kernel iters: 256
```

```
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 221.3
  Split approx (ms): H2D 87.6567 | Kernel 52.0198 | D2H 81.6664
Async end-to-end (ms): 116.889
Speedup (sync/async): 1.89325x
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 67108864 (64 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 221.257
  Split approx (ms): H2D 87.9045 | Kernel 50.0921 | D2H 81.6309
Async end-to-end (ms): 119.318
Speedup (sync/async): 1.85436x
```

Stream:

```
!./overlap --bytes 1073741824 --chunk 33554432 --streams 1 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
!./overlap --bytes 1073741824 --chunk 33554432 --streams 2 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
!./overlap --bytes 1073741824 --chunk 33554432 --streams 4 --iters 256 --pinned 1 --warmup 1 --reps 5 --verify 0
```

```
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 1
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 220.662
  Split approx (ms): H2D 87.5486 | Kernel 52.362 | D2H 81.6726
Async end-to-end (ms): 215.951
Speedup (sync/async): 1.02181x
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 2
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5

=== Results (median over reps) ===
Sync end-to-end (ms): 221.753
  Split approx (ms): H2D 87.6144 | Kernel 51.8588 | D2H 81.6487
```

```
Async end-to-end (ms): 127.335
Speedup (sync/async): 1.74149x
GPU: Tesla T4
Total bytes: 1073741824 (1024 MiB)
Chunk bytes: 33554432 (32 MiB)
Streams: 4
Kernel iters: 256
Host pinned: 1
Warmup: 1, reps: 5
```

=== Results (median over reps) ===

```
Sync end-to-end (ms): 221.222
  Split approx (ms): H2D 87.7031 | Kernel 51.4645 | D2H 81.638
Async end-to-end (ms): 116.47
Speedup (sync/async): 1.89939x
```

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.